

## Algoritmo de Planificación Round-Robin

Sistemas Operativos

Actividad 10 – Programa 5

**Alumno:** Quijas Contreras Victor Manuel

**Código:** 220429771

**Carrera:** ICOM **Sección:** D05

**Profesor:** Becerra Velázquez Violeta Del Rocío

**Departamento:** Ciencias Computacionales

**Fecha:** 25/04/2026

## Índice

|                                |   |
|--------------------------------|---|
| Índice .....                   | 1 |
| Desarrollo .....               | 1 |
| Refactorización.....           | 1 |
| Eventos.....                   | 1 |
| Iteraciones .....              | 2 |
| Ejecución de los eventos ..... | 2 |
| Aplicación de Round-Robin..... | 2 |
| Conclusión .....               | 4 |

## Desarrollo

### Refactorización

Para la realización de este programa se decidió por realizar una refactorización del código para facilitar la implementación de nuevas funciones que alteren su comportamiento. Para esto se optó por la realización que el sistema reaccionara en base a eventos, cambiar la lógica de cómo se ejecutan las iteraciones y agregar el evento de Round-Robin.

Anteriormente cada ciclo de reloj se producía cada segundo, pero cada evento se ejecutaba de manera diferente, por ejemplo: cuando un evento había terminado de manera natural, su tiempo de finalización se registraba un segundo después, pero si este terminaba por error, el tiempo de finalización se guardaba en esa misma iteración, pero se incluía a la tabla hasta el siguiente segundo. Estas maneras de registrar los tiempos generaban inconsistencias en el registro de tiempos. La solución a esto se desarrolló de la forma:

### Eventos

Ahora cada acción que realiza el sistema operativo se desgloza en varios eventos principales que a su vez se descomponen en métodos más simples. Para llamar a un evento, el sistema no lo ejecuta directamente, sino que lo agrega a una cola de eventos pendientes a ejecutar. Esta cola se ejecuta cada iteración, y ejecuta los eventos por orden de llegada. Una vez que se comienzan a ejecutar los eventos de la cola, esta se vacía y los agrega a una lista auxiliar con la que comenzará a ejecutar cada uno de los eventos pendientes, con el fin de evitar que un evento se quede sin ejecutar. Debido

## Algoritmo de Planificación Round-Robin

a esto, si se hubiera quedado con las ejecuciones anteriores, tardaría el programa demasiado en responder a alguna petición. Por lo que se optó por cambiar la lógica de cómo opera cada iteración.

### Iteraciones

Ahora cada iteración deja de simular en segundo de paso y se convierte en una ráfaga de iteraciones. Cada repetición se realiza de la manera más rápida que la biblioteca Swing de Java nos lo permite, que es alrededor de 15 ms cada vuelta. Con el objetivo de que los eventos en el programa se realicen lo más rápido posible y no tenga una alta latencia a la hora de interactuar con el programa. Esto a su vez beneficia al registrar los tiempos pues las entradas y salidas de los procesos en el procesador se realizan en tiempo real y de manera uniforme.

Para que el contador global del programa avance cada segundo, se operan con milisegundos reales del sistema, por lo tanto, es más preciso en sus tiempos de ejecución.

### Ejecución de los eventos

Cada vez que el contador detecta que ha pasado justamente un segundo el programa realiza tres tareas específicas.

1. Incrementar en uno el contador global.
2. Permitir que el programa se ejecute en la siguiente iteración si en la cola de eventos no existe registrada la salida de un proceso.
3. Ejecutar la cola de eventos.
4. Lanzar eventos obligatorios.

Ahora el contador global se aumenta desde un inicio pues ya no existen conflictos en las asignaciones de los tiempos como anteriormente se mencionaba. También es necesario que el programa valide si el procesador no va a sacar a el proceso que tiene en ejecución, para poder encolar un evento de ejecución principal. Después de ejecutar el programa, ejecuta toda la cola de eventos que se han encolado en esa iteración para, posteriormente, lanzar los eventos complementarios obligatorios que se deben de ejecutar. Que para este programa son:

- Ingresar de la lista de bloqueados a la memoria.
- Verificar si el quantum se ha cumplido para sacar al proceso en ejecución del procesador.

### Aplicación de Round-Robin

Para aplicar el algoritmo de ejecución llamado Round-Robin necesitamos crear dos variables nuevas. Una para guardar el valor del quantum que se quiere cumplir y otra para contar cuantas iteraciones lleva ese proceso.

Como se mencionaba, la implementación de nuevas funciones para el procesador se facilita. Round-Robin no es más que un evento compuesto por métodos que ya están contruidos por lo que solo hace falta agregar su funcionalidad y especificar que cada vez que un proceso vaya a salir de ejecución, el contador de las iteraciones va a reiniciarse pues, cada vez que un proceso sale de ejecución el contador del quantum se reinicia. De esta forma queda integrado el método ejecución Round-Robin para nuestro programa.

```
private Proceso SacarDeEjecucion() {  
    if (procesador.isEmpty())  
        return null;  
    Proceso enEjecucion;  
    enEjecucion = procesador.get(0);  
    procesador.remove(0);  
    contadorQuantum = 0;  
    return enEjecucion;  
}
```

```
private void RR() {  
    if (procesador.isEmpty())  
        return;  
    Proceso loSaca;  
    loSaca = SacarDeEjecucion();  
    memoria.add(loSaca);  
    IngresarAEjecucion();  
    RefrescarTabla();  
}
```

## **Algoritmo de Planificación Round-Robin**

### **Conclusión**

El algoritmo de planificación de Round-Robin es un algoritmo eficiente dependiendo de la duración del quantum y de la longitud del servicio que se espera que tengan cada uno de los procesos. Su aplicación a el algoritmo anterior que era FCFS es bastante sencilla pues solo altera la forma en que se ejecuta cada ciclo al validar si la condición del quantum ya se cumplió para realizar su método apropiativo de sacar el proceso de ejecución para mandarlo a la memoria.